

Ejercitación Hash Tables

Algoritmos y Estructuras de Datos II

Alumno: Matías Cúneo

Parte 1

1. El tiempo de acceso promedio en esta tabla sería de $\theta(1 + \frac{n}{m}) = \theta(2)$.

Slot 1 (mod = 0): 5

Slot 2 (mod = 1): 28 -> 19 -> 10

Slot 3 (mod = 2): 20

Slot 4 (mod = 3): 12

Slot 5 (mod = 4):

Slot 6 (mod = 5):

Slot 7 (mod = 6): 15 -> 33

Slot 8 (mod = 7):

Slot 9 (mod = 8): 17

Parte 2

3. $h(k) = \lfloor 1000(k \frac{\sqrt{5}-1}{2} \bmod 1) \rfloor$

- $h(61) = 700$
- $h(62) = 318$
- $h(63) = 936$
- $h(64) = 554$
- $h(65) = 172$

4. La solución propuesta involucra hashear todos los caracteres de la primera lista y luego eliminar todos los caracteres hashados de la segunda lista en la misma tabla hash, si eran una permutación, la tabla debería terminar vacía. Se recorre toda la primer cadena y segunda cadena (misma longitud si son permutación, n) y se itera toda la tabla hash (longitud 26), entonces la complejidad es de:

$$O(2n + 26) = O(2n) = O(n)$$

5. El algoritmo genera una tabla hash del tamaño de la lista, luego la itera para llenar la tabla y finalmente recorre la tabla para ver si hubo colisiones, efectivamente recorriendo la longitud n de la lista dos veces, luego su complejidad es de: $O(2n) = O(n)$.

6. Para este ejercicio se utilizó la función de hash universal, ya que con una función de módulo simple, por la naturaleza de la distribución de los códigos postales, se podría observar un agrupamiento de entradas por zona.
7. Ambas soluciones propuestas cuentan con una complejidad temporal de $O(n)$, ya que, para el algoritmo sin hash, se itera a través de la cadena una sola vez, y para la implementación con hash se recorre la cadena una vez y luego la tabla, resultando en: $O(n + 122) = O(n)$.
8. La solución propuesta involucra pre procesar la cadena en una lista de prefijos hasheados, entonces las partes principales del algoritmo son: preprocesar la cadena principal $O(n)$, hashear la subcadena $O(m)$ e iterar mediante ventanas de tamaño m la cadena principal $O(n - m + 1)$, entonces la complejidad del algoritmo completo se reduce a:

$$O(n + m + (n - m + 1)) = O(2n + 1) = O(n)$$
9. El caso promedio y general para el algoritmo propuesto, que primero crea una tabla hash con todos los valores de T , en $O(|T|)$, y luego itera S buscando sus valores en la tabla es de: $O(|T| + |S|)$.

Parte 3

10.

Tabla \ Función	Linear Probing	Quadratic Probing	Double Hashing
Slot 1	22	22	22
Slot 2	88		
Slot 3		88	59
Slot 4		17	17
Slot 5	4	4	4
Slot 6	15		15
Slot 7	28	28	28
Slot 8	17	59	88
Slot 9	59	15	
Slot 10	31	31	31
Slot 11	10	10	10

12. La tabla resultante es la tabla (C), ya que se está implementando linear probing esencialmente con la función $h(k) = ((k \bmod 10) + i) \bmod m$. Este es el resultado de implementar linear probing con esa función de hash con el código del ejercicio 10:

```
12: slot 2
18: slot 8
13: slot 3
2: slot 4
3: slot 5
23: slot 6
5: slot 7
15: slot 9
```

13. La opción (C) también, ya que al aplicar la operación módulo 10 a k , hacemos que solo el último dígito valga como key, entonces para este ejercicio y el anterior, el orden de inserción va a ser o el slot indicado por el último dígito de k , o el primer slot libre después de ese.